MIT 8103 · ADVANCED DATABASE SYSTEMS

University/Student Result Portal

Technical Architecture README & Portfolio Evaluation Summary

Author: Samuel Chukwuma **Student ID:** 301910606 **Engines:** PostgreSQL & MongoDB

Source Repository: https://github.com/Weltraum-Fahrer/Chukwuma_Samuel_301910606

01 System Overview & Operational Scope

The **University/Student Result Portal** is a hybrid relational and document-oriented database system engineered to manage academic assessment records with strict data integrity and auditable event history. The architecture serves two primary user groups—**students** reviewing their published grades and **academic staff** recording or amending course marks—across three core operational workflows:

- **Result Recording:** Persist individual student scores against specific scheduled course offerings with automatic timestamps.
- **Targeted Retrieval:** Query and audit recorded results filtered by student identifier, course code, and academic term.
- **Analytical Aggregation:** Compute result counts and mean scores across course offerings and cohorts.

02 Relational Schema & Integrity Modelling (Portfolio 1)

The transactional core is normalized through **1NF, 2NF, and 3NF** across four decoupled tables—`students`, `courses`, `offerings`, and `results`—eliminating repeating groups and update anomalies. Primary keys enforce entity uniqueness, foreign keys guarantee referential integrity across parent-child relationships, and domain `CHECK` constraints enforce business invariants, specifically requiring every offering's start date to precede or equal its end date (`starts_on <= ends_on`). During negative constraint testing, all **13 deliberately invalid inserts** were rejected cleanly without mutating valid baseline records.

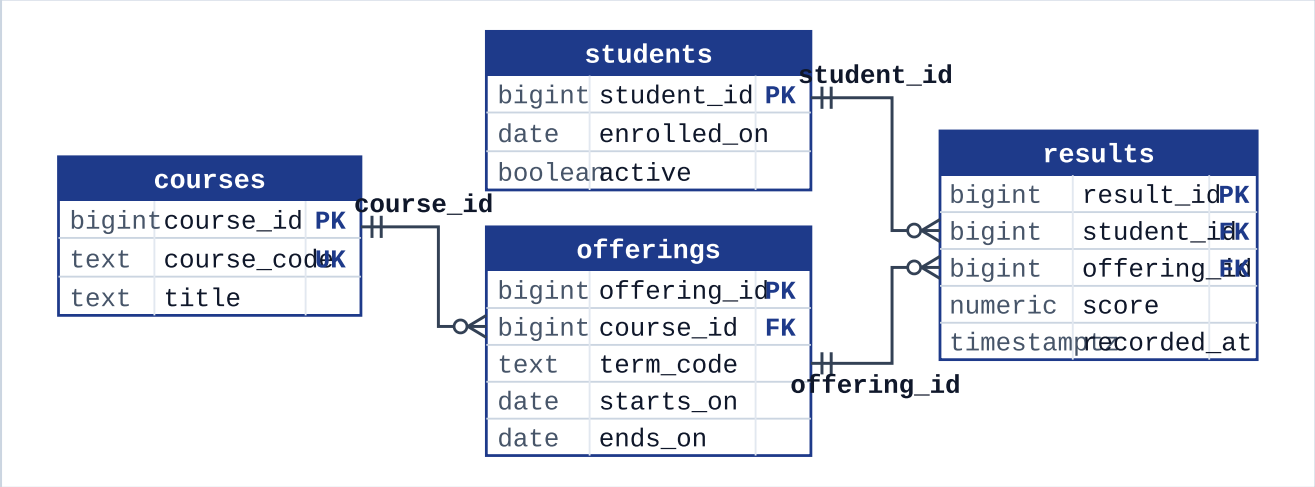


Figure 1. Relational Entity-Relationship Schema. Crow's-foot notation illustrates mandatory-one (`||`) to optional-many (`o<`) cardinality across primary keys (`PK`), unique keys (`UK`), and foreign keys (`FK`).

Table	Key Constraints	Domain Role & Enforced Invariants
`students`	`PK: student_id`	Stores student enrollment dates (`enrolled_on`) and account status (`active`).
`courses`	`PK: course_id` `UK: course_code`	Defines canonical catalog courses with unique institutional codes and titles.
`offerings`	`PK: offering_id` `FK: course_id`	Binds courses to academic terms (`term_code`); enforces `starts_on <= ends_on` .
`results`	`PK: result_id` `FK: student_id, offering_id`	Records numeric assessment scores and `timestamptz` audit timestamps; 13 invalid inserts blocked.

03 Query Processing & Index Optimisation (Portfolio 2)

Three representative SQL queries were executed against the sample dataset to validate multi-table joins and aggregations; the relational join returned **2 rows**, confirming linkage between term `2026FA` and course `SYN101` . To benchmark physical access paths, a dedicated dataset of **20,000 performance rows** was evaluated before and after index creation. Introducing the targeted index reduced median lookup latency from **0.944 ms** to **0.181 ms** while returning identical result sets, with `EXPLAIN` plans confirming direct index utilization in the tested environment.

Evaluation Phase	Dataset Scale	Observed Metric	Verification Outcome
Functional SQL Suite (3 Queries)	Normalized sample records	2 rows returned on join	Confirmed `2026FA` paired with `SYN101`
Baseline Lookup (Pre-Index)	20,000 performance rows	0.944 ms median latency	Baseline reference result set captured
Indexed Lookup (Post-Index)	20,000 performance rows	0.181 ms median latency	Identical rows; `EXPLAIN` confirmed index scan

04 Transaction Isolation & Concurrency Control (Portfolio 3)

ACID isolation and write-conflict behavior were verified using a transient counter attribute on `results` , strictly isolated to this concurrency exercise so core business columns remained untouched. Under PostgreSQL's default `READ COMMITTED` isolation level, uncommitted modifications remained invisible to concurrent sessions, and `ROLLBACK` cleanly restored the prior committed state. Under concurrent read-modify-write contention, unsynchronized sessions suffered a lost update (final value: **2**), whereas pessimistic row locking via `SELECT FOR UPDATE` forced Session B to block until Session A committed, preserving both increments (final value: **3**).

Concurrency Scenario	Execution Behavior	Outcome / Final State
`READ COMMITTED` & `ROLLBACK`	Session A modified counter without committing; Session B queried concurrently before rollback.	Dirty read prevented; `ROLLBACK` restored prior committed value.
Concurrent Update (Unlocked)	Two sessions read the same initial counter concurrently and applied separate increments.	Lost-update anomaly occurred; final counter left at **2**.
Pessimistic Locking (`SELECT FOR UPDATE`)	Session A locked row before read; Session B waited for Session A's `COMMIT` .	Serialized execution preserved both increments; final counter reached **3**.

05 Polyglot Persistence & NoSQL Event Modelling (Portfolio 4)

To complement strict relational storage with flexible activity auditing, domain lifecycle events were modeled in MongoDB using a `result_events` collection. Four fictional sample event documents were inserted and validated across four targeted queries. Collection-level schema validation enforced structural rules by rejecting both an **invalid date type** and a **duplicate event identifier**. Each document pairs embedded entity snapshots—keeping contextual student and course metadata co-located for single-read efficiency—with a polymorphic `details` sub-document whose schema adapts by event type, while PostgreSQL remains the authoritative relational system of record.

06 Distributed Topology & Cloud Availability (Portfolio 5)

For high availability of the `result_events` workload in cloud deployments, a **three-member MongoDB replica set** distributed across separate failure zones was specified. The architectural analysis addresses automated primary elections during node outages, `majority` write acknowledgement trade-offs between durability and commit latency, secondary replication lag on eventual-consistency reads, and horizontal sharding considerations as event volume scales. Crucially, replication provides operational continuity rather than point-in-time disaster recovery—requiring dedicated backup pipelines alongside a separate high-availability plan for PostgreSQL. In accordance with the permitted architectural discussion scope, no live replica set or failover test was deployed.

07 Principal Engineering Reflection

- **Storage-Boundary Invariants:** Decomposing entities into `students` , `courses` , `offerings` , and `results` eliminates redundant updates, while declarative constraints (`FK` links and `starts_on <= ends_on`) prevent invalid state from ever entering persistent storage.
- **Serializing Read-Modify-Write Paths:** Under `READ COMMITTED` , concurrent transactions reading shared state before commit risk silent overwrites; acquiring explicit row locks (`SELECT FOR UPDATE`) prior to reading enforces strict write serialization.
- **Execution Plans Over Output Equivalence:** Identical query results can conceal order-of-magnitude differences in I/O and CPU cost; inspecting `EXPLAIN` plans is mandatory whenever validating indexing decisions.

08 Attestation

> I designed, structured, and verified the database architecture, normalization progression, constraint rules, and engineering evaluations presented in this portfolio. The relational schema design, business invariant definitions, query benchmarking criteria, concurrency test scenarios, and architectural trade-off analyses reflect my own technical reasoning and direct review of every artifact in the repository.
>
> To streamline repetitive implementation mechanics, I used **GitHub Copilot** and the **portfolio generator** as assistive tools while drafting boilerplate SQL/JavaScript scripts and initial prose explanations. The portfolio generator also executed the PostgreSQL and MongoDB test scripts, verified their outputs, and captured the supporting result screenshots. I personally audited all generated scripts, validated every query plan and constraint rejection against expected behavior, and take full responsibility for the correctness and defense of the submitted work.
>
> ---
>
> **Samuel Chukwuma · Student ID: 301910606 · MIT 8103 Advanced Database Systems**

09 Repository Evidence & Technical References

Complete SQL DDL/DML scripts, MongoDB validation code, `EXPLAIN` execution plans, tabular query outputs, and captured screenshots are archived in the portfolio folders at https://github.com/Weltraum-Fahrer/Chukwuma_Samuel_301910606.

Technology Domain	Canonical Documentation & Standards Referenced
PostgreSQL	Constraints (`PRIMARY KEY` , `FOREIGN KEY` , `UNIQUE` , `CHECK`); Query Execution Plans (`EXPLAIN`); Transaction Isolation (`READ COMMITTED`); Explicit Row-Level Locks (`SELECT FOR UPDATE`).
MongoDB	Data Modelling & Embedded Document Patterns; Replica Sets & Elections; Horizontal Sharding Architecture.
Diagramming	Mermaid Entity-Relationship (ER) Diagrams & Crow's-Foot Cardinality Notation.